

Military Institute of Science and Technology
Department of Electrical Electronic and Communication Engineering



Course Name: Electrical Measurement, Instrumentation and Sensors Laboratory

Course Code: EEE-314

Experiment Name: Measurement of Unknown Resistance Using a 555 Timer Astable Multivibrator Interfaced with Arduino.

Course Conducted By:

1.Lt Col Md Aminul Islam
2.Asst.Prof.A.A.M.Shah Sadman
3.Lec Nazifa Nawar
4.Lec Raian Habib Himel
5.Lec Zubaid Azad Shikha

Group No: 02

Group Members:

Name	Student ID	Contribution
Tahiya Iffat Majumder	202416002	Report & Documentation
Ragib Mahmud	202416012	Hardware Assembly & Testing
Mahasin Mukit Samiha	202416024	Report & Documentation
Md Abdullah Al Nayeem	202416036	Hardware Assembly & Testing
Md.Mehedi Hasan Khan Apurbo	202416047	Proteus Design & Simulation
Md.Mymunun Kabir Munna	202316209	Proteus Design & Simulation

MILITARY INSTITUTE OF SCIENCE & TECHNOLOGY
EEE 314 | MEASUREMENT AND INSTRUMENTATION LABORATORY

PROJECT NAME: Measurement of Unknown Resistance Using a 555 Timer Astable Multivibrator Interfaced with Arduino.

OBJECTIVE:

The objective of this project is to design, simulate and implement a resistance measurement system that uses a 555 timer IC operating in astable mode to convert an unknown resistance into a proportional time period, which is then captured and averaged over 20 consecutive cycles, processed and displayed by an Arduino Uno microcontroller on a 16×2 LCD as an auto-scaled resistance value (Ω , $k\Omega$ or $M\Omega$) together with the measured period.

THEORY:

The 555 timer is one of the most widely used ICs in electronics, valued for its robustness, low cost and stable timing behavior. It derives its name from the three internal 5 k Ω resistors that set the reference voltages of its two internal comparators. The 555 can operate in three modes :astable, mono-stable and, bi-stable. In this project, the astable mode is used, since the output period of this mode can be made a linear function of an external resistor, which allows an unknown resistance to be inferred purely from a time measurement.

Astable Operation:

When connected in astable mode with a fixed resistor R_f (labelled R_1 in the circuit diagram), an unknown resistor R_x (labelled R_x in the diagram) and a timing capacitor C (C_1 in the diagram), the capacitor charges through $(R_f + R_x)$ and discharges through R_x alone, between the $\frac{1}{3}V_{CC}$ and $\frac{2}{3}V_{CC}$ comparator thresholds. This gives-

$$t_{HIGH} = 0.693 (R_f + R_x)C \quad (1.1)$$

$$t_{LOW} = 0.693 R_x C \quad (1.2)$$

$$T = t_{HIGH} + t_{LOW} = 0.693 (R_f + 2R_x)C \quad (1.3)$$

$$f = \frac{1.44}{(R_f + 2R_x)C} \quad (1.4)$$

Where T is the period of the square wave observed at pin 3 (OUTPUT) of the 555. Since R_f and C are fixed and known, measuring the period T of the oscillator directly gives the unknown resistance R_x :

$$R_x = \frac{1.44T}{2C} - \frac{R_f}{2} \quad (1.5)$$

The Arduino measures T by time stamping the RISING edges of the 555 output (pin 3) on the external interrupt pin D2 using the **micros()** function. Instead of relying on a single period, the interrupt service routine counts 20 consecutive cycles (CYCLES = 20) and stores the total time elapsed between the first and the last edge; dividing this total by 20 gives the average period T in microseconds, which reduces the effect of the 4 μs resolution of **micros()** and of cycle-to-cycle jitter. The average period is converted to seconds and substituted into the resistance equation, which the firmware evaluates in the equivalent form $R_x = \frac{T}{2C \ln 2} - \frac{R_f}{2}$ with $R_f = R_1 = 100 \Omega$, $C = 10 \mu F$ and $\ln 2 = 0.69314718$, to compute the resistance in real time.

The result is shown on the 16×2 LCD: line 1 displays the resistance auto-scaled to Ω , $k\Omega$ or $M\Omega$, and line 2 displays the averaged period in μs . In the Proteus simulation, the 555 output is

also connected to channel A of a virtual oscilloscope, so the square wave and its period can be observed directly. Because a new reading is produced only after 20 oscillator cycles, one reading takes $20T$. The table below lists the expected period and the time per reading for representative values of R_x ($R_x = 4\text{ k}\Omega$ is the value used in the simulation).

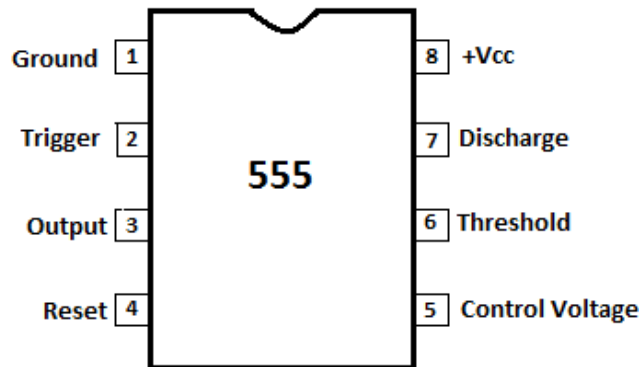


Fig 1: Pin Diagram of 555 Timer IC

CIRCUIT DIAGRAM:

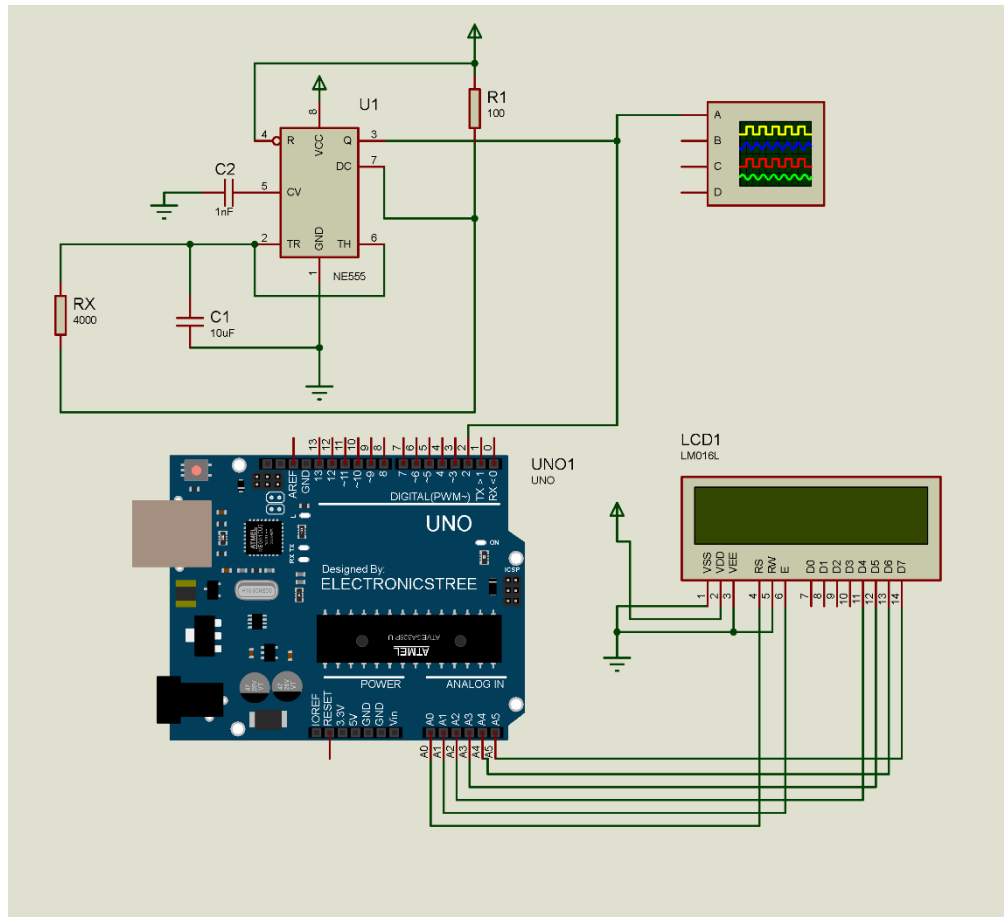


Fig 2: Proteus schematic of the Arduino 555 resistance measurement circuit ($R_x = 4\text{ k}\Omega$ in the simulation)

REQUIRED COMPONENTS:

Serial No.	Name of The Component	Quantity
1)	Arduino Uno	1
2)	NE555 precision timer IC	1
3)	16×2 Alphanumeric LCD	1
4)	Resistor (100 Ω)	2
5)	Test Resistor (22 Ω -100K Ω)	As required
6)	Oscilloscope	1
7)	Capacitor (10uf, 1 nF)	2
8)	Breadboard	As Required
9)	5V DC Supply	1
10)	Digital Multimeter	1
11)	Proteus Design Suite	1
12)	Connecting Wires	1

CONNECTIONS:

The circuit was designed and simulated in Proteus Design Suite, using an Arduino Uno R₃ (UNO₁), an NE555 timer (U₁), a 16×2 LM016L alphanumeric LCD (LCD₁) and a virtual oscilloscope. The 555 is wired as an astable oscillator with a fixed resistor R₁ = 100 Ω , the resistor under test R_X (set to 4 k Ω in the simulation), a 10 μ F timing capacitor C₁ and a 1 nF control-voltage capacitor C₂, exactly as required by the period equation derived above. The 555 output (pin 3) drives the Arduino's external interrupt pin D2 and channel A of the oscilloscope, and the LCD is wired to the Arduino's analogue pins A0–A5 as summarised below.

555 / LCD Pin	Connected To	Function
555 Pin 1 (GND)	Ground rail	Ground reference
555 Pin 2 (TRIG)	Shorted to Pin 6 (NODE B)	Trigger input
555 Pin 3 (OUT)	Arduino D2 and oscilloscope channel A	Square-wave output → external interrupt (RISING)
555 Pin 4 (RESET)	+5 V	Held high, reset disabled
555 Pin 5 (CTRL)	C ₂ (1 nF) to GND	Noise bypass on internal reference
555 Pin 6 (THR)	Shorted to Pin 6 (NODE B)	Threshold input
555 Pin 7 (DISCH)	Node A (junction of R ₁ and R _X)	Discharge path
555 Pin 8 (VCC)	+5 V	Supply
R ₁ (100 Ω)	+5 V ↔ Node A	Fixed charging resistor
R _X (unknown)	Node A ↔ Node B	Resistor under test
C ₁ (10 μF)	Node B ↔ GND	Timing capacitor
LCD RS / E	Arduino A0 / A1	Register-select / Enable
LCD D4–D7	Arduino A2 / A3 / A4 / A5	4-bit data bus
LCD D0–D3	Not connected	Unused (4-bit mode)
LCD R/W, VSS, VEE	GND	Write mode, ground, contrast pin tied to GND (maximum contrast)
LCD VDD	+5 V	Supply
Oscilloscope channel A	555 Pin 3 (OUT)	Waveform / period monitoring (channels B–D unused)

Because the Arduino I/O pins are 5 V-tolerant, the 555 is powered from the Arduino's regulated +5 V rail (rather than the 9 V supply typically used with a 555 alone) so that its output can be fed directly into digital pin D2 without a level-shifting or clamping network.

WORKING PRINCIPLE:

- 1) On power-up, the LCD is initialized in 16×2 mode, pin D2 is configured as an input and the interrupt service routine (ISR) `measurePeriod()` is attached to it, triggered on the RISING edge of the 555 output. The LCD displays “Resistance Meter Ready” for 1.5 s and is then cleared.
- 2) Each time a rising edge occurs, the ISR reads `micros()`. The first edge is stored as `firstEdge`; edges arriving less than 1000 μs after it are ignored (a simple guard against spurious edges), while every later edge is stored as `last Edge` and counted as one cycle.
- 3) When 20 cycles (CYCLES = 20) have been counted, the ISR stores **totalTime = lastEdge – first Edge**, sets the new Measurement flag, and restarts the next 20-cycle measurement from the current edge.
- 4) The main `loop()` copies total Time and the flag with interrupts briefly disabled (to avoid a torn read) and clears the flag. If no new measurement is ready, `loop()` simply returns, so the LCD is refreshed only when a fresh 20-cycle result is available.
- 5) The average period is calculated as $T = \frac{\text{Total Time}}{20}$ converted to seconds. The resistance is then computed find $R_x = \frac{T}{2C \ln 2} - \frac{R_f}{2}$ with a floor of 0 Ω to prevent a negative reading from noise on very small resistances.
- 6) The LCD is cleared and updated: line 1 shows “R=” followed by the resistance auto-scaled to Ω, KΩ or MΩ (2 decimal places for ohm, 3 for KΩ and MΩ), and line 2 shows “T=” followed by the averaged period in μs (2 decimal places). A 200 ms delay follows before the next update.

PROCEDURE:

- 1) Build the circuit shown above in Proteus and on breadboard for hardware implementation, keeping $R_1 = 100\ \Omega$, $C_1 = 10\ \mu\text{F}$ and $C_2 = 1\ \text{nF}$ fixed and leaving the R_x position free for the resistor to be tested.
- 2) Compile the Arduino sketch in the Arduino IDE and load the generated .hex file into the Arduino UNO component's program property in Proteus (or upload it to a physical Arduino Uno).
- 3) Run the simulation or power the circuit and confirm that the LCD shows the start-up message “Resistance Meter Ready”, and that the resistance and period readings appear after the first 20 oscillator cycles (for $R_x = 4\ \text{K}\Omega$ the LCD should show approximately $R=4.000\text{kohm}$ and $T=56145\mu\text{s}$). Check on oscilloscope channel A that the 555 output is a square wave of the same period.
- 4) Using a digital multimeter, measure and note the actual value of a test resistor, then insert it at the R_x position.
- 5) Allow the reading to settle (wait for at least one full 20-cycle measurement after every change of R_x), then note the resistance R on LCD line 1 and the period T on LCD line 2. Optionally compare T with the period observed on the oscilloscope.
- 6) Repeat step 4–5 for at least seven resistors spanning the measurable range (e.g. approximately 47 Ω, 100 Ω, 470 Ω, 1 KΩ, 4.7 KΩ, 10 KΩ and 47 KΩ).
- 7) Tabulate the actual resistance values from multimeter reading, the measured period, the resistance calculated by the Arduino and the percentage error for each trial.

DATA TABLE FOR HARDWARE:

Obs.	Actual R (multimeter)	Measured Period T	Calculated Rx	% Error
1	100 Ω	2166.78 μs	106.30 Ω	6.30%
2	1 K Ω	15207.65 μs	1047.00 Ω	4.70%
3	10 K Ω	144035.98 μs	10340.00 Ω	3.40%
4	20 K Ω	285992.53 μs	20580.00 Ω	2.90%
5	100 K Ω	1416099.69 μs	102100.00 Ω	2.10%
6	1 M Ω	14085443.86 μs	1016000.00 Ω	1.60%

DATA TABLE FOR SOFTWARE:

Obs.	Actual R (multimeter)	Measured Period T	Calculated Rx	% Error
1	100 Ω	2083.32 μs	100.28 Ω	0.28%
2	1 K Ω	14564.41 μs	1000.60 Ω	0.06%
3	10 K Ω	139343.38 μs	10001.50 Ω	0.015%
4	20 K Ω	277974.20 μs	20001.60 Ω	0.008%
5	100 K Ω	1387029.10 μs	100003.00 Ω	0.003%
6	1 M Ω	13863775.39 μs	1000010.00 Ω	0.001%

EXPECTED VALUES FOR DIFFERENT Rx:

Actual Rx	Expected Period T
47 Ω	1344.71 μs
100 Ω	2079.44 μs
1 k Ω	14556.09 μs
4 k Ω (simulated)	56144.92 μs
10 k Ω	139322.58 μs
47 k Ω	652251.50 μs
100 k Ω	1386987.51 μs

REPORT QUESTIONS:

- 1) Complete data table for at least seven resistor values spanning the measurable range.
- 2) Percentage error calculation for each trial: $\%Error = \frac{R_{measured} - R_{actual}}{R_{actual}} \times 100\%$
- 3) Write short explanation of the three modes of operation of the 555.
- 4) Derive the relationship between the measured period and the unknown resistance in this experiment
- 5) What are the functions of the trigger, threshold, discharge, and output pins of the 555 timer in this circuit?
- 6) What happens to the output frequency when R_X is increased?

PRECAUTIONS:

- Power the 555 from the Arduino's regulated +5 V rail, not a 9 V supply, since its output feeds an Arduino digital pin directly.
- Include the 1 nF capacitor (C2) on pin 5 (CTRL) to reduce noise-induced jitter in the period measurement.
- Connect the LCD exactly as declared in the firmware, LiquidCrystal lcd(A0, A1, A2, A3, A4, A5), i.e. RS = A0, E = A1 and D4–D7 = A2–A5, with R/W tied to GND.
- After changing R_X , wait for at least one full 20-cycle measurement (about $20 \times T$) before reading the display, since the first result may include cycles from the previous resistor.
- Verify tight breadboard connections — loose contacts add series resistance that is significant for low-value test resistors.
- Avoid very large R_X values: each reading needs 20 oscillator cycles (about 277 s at 1 M Ω), which is inconvenient to observe.
- Avoid resistors below roughly 22 Ω , where the oscillator period is shorter than the 1000 μ s edge guard in the ISR and the reading is over-estimated.

APPENDIX

APPLICATIONS:

- Low-cost digital ohmmeter for teaching and bench troubleshooting.
- Demonstration of RC timing principles and 555 astable operation in an embedded-systems context.
- Template for other 555-based sensor-to-microcontroller interfacing projects (e.g. capacitance or light-dependent resistance measurement).
- The following table compares our Arduino–555 timer resistance measurement system with the Wheatstone bridge and loss-of-charge methods.

ADVANTAGES OF THE PROPOSED ARDUINO–555 TIMER METHOD OVER WHEATSTONE BRIDGE AND LOSS-OF-CHARGE METHODS:

Parameter	Arduino–555 Method	Wheatstone Bridge	Loss-of-Charge Method
Measurement principle	Measures resistance from the time period of a 555-timer oscillator.	Measures resistance by balancing a bridge circuit.	Measures resistance from the capacitor discharge rate.
Circuit complexity	Simple and compact circuit.	Requires a bridge network and balancing/detection circuit.	Requires charging, discharging, and voltage-measurement circuitry.
Measurement process	Fully automatic using Arduino.	May require manual balancing or extra detection circuitry.	Requires careful timing and voltage monitoring.
Oscilloscope compatibility	The 555 output is a continuous square wave, so its time period can also be measured directly using an oscilloscope	Does not normally provide a continuous oscillating output for direct time period measurement.	Does not normally provide a continuous periodic waveform; the discharge is a one-time voltage decay that must be measured over time.
Display	Direct digital reading on an LCD.	Usually needs an additional meter or calculation.	Needs additional processing and display.
Speed	Continuous automatic measurement; a new reading is displayed after every 20 oscillator cycles.	May require adjustment for different resistance values.	Can be slow, especially for high resistance.
Ease of use	Easy to operate and suitable for beginners.	Comparatively more difficult to operate and calibrate.	More sensitive to Capacitor leakage and timing errors.
Noise reduction	Averages 20 consecutive periods to give stable, low-jitter reading.	No built-in averaging; the result depends on how well the null is detected.	Each reading is a single decay, so noise is not averaged out.

Conclusion:

Our Arduino-555 method is preferred for this project because it is simple, low-cost, automatic, and provides direct digital readings. In addition, the 555 produces a continuous square-wave output whose time period can be measured by an oscilloscope as well as by Arduino. The Wheatstone bridge and loss-of-charge methods do not provide this same continuous periodic output for direct time-period measurement.

ARDUINO FIRMWARE

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(A0, A1, A2, A3, A4, A5); const byte signalPin = 2;
const float R1 = 100.0; const float C = 10e-6;
const float LN2 = 0.69314718056;
volatile unsigned long firstEdge = 0; volatile unsigned long lastEdge = 0; volatile unsigned long totalTime = 0; volatile
bool newMeasurement = false;
const byte CYCLES = 20; void measurePeriod()
{
    unsigned long now = micros();

    if (firstEdge == 0)
    {
        firstEdge = now;
    }
    else if (now - firstEdge >= 1000)
    {
        lastEdge = now;

        // Measure 20 cycles
        static byte count = 0;
        count++;

        if (count >= CYCLES)
        {
            totalTime = lastEdge - firstEdge;
            newMeasurement = true;

            firstEdge = now; count
            = 0;
        }
    }
}

void setup()
{
    lcd.begin(16, 2);

    pinMode(signalPin, INPUT);

    attachInterrupt(digitalPinToInterrupt(signalPin), measurePeriod, RISING);

    lcd.print("Resistance");
    lcd.setCursor(0, 1);
    lcd.print("Meter Ready");

    delay(1500); lcd.clear();
}

void loop()
{
    unsigned long time; bool
    ready;
```

```

noInterrupts(); time =
totalTime;
ready = newMeasurement;
newMeasurement = false; interrupts();

if (!ready)
    return;

// Average period in microseconds float
T_us = time / (float)CYCLES;

// Convert to seconds float
T = T_us * 1e-6;

// Calculate Rx
float Rx = T / (2.0 * LN2 * C) - R1 / 2.0;

if (Rx < 0) Rx
    = 0;

// LCD
lcd.clear();

lcd.setCursor(0, 0);
lcd.print("R=");

if (Rx >= 1000000)
{
    lcd.print(Rx / 1000000.0, 3);
    lcd.print("Mohm");
}
else if (Rx >= 1000)
{
    lcd.print(Rx / 1000.0, 3);
    lcd.print("kohm");
}
else
{
    lcd.print(Rx, 2);
    lcd.print("ohm");
}

lcd.setCursor(0, 1);
lcd.print("T=");
lcd.print(T_us, 2);
lcd.print("us");

delay(200);
}

```

